



INDUSTRIAL TRAINING REPORT

Submitted in partial fulfilment for the

Degree of Bachelor of Technology,
Engineering Physics

Training Organisation: Cadence Design Systems (India) Pvt. Ltd., Noida

Team: Product Engineering – Voltus (Power Integrity Signoff)

Project: Distributed Automation for Voltus Power-Integrity Signoff

Duration: 1 June 2026 to 31 July 2026

Submitted by:

ISHAAN SINGHAL

23/EP/047

Department of Applied Physics, Delhi Technological University,
Bawana Road, Delhi-110042

Contents

1	Acknowledgement	3
2	Introduction	4
2.1	About the Organization	4
2.2	About the Team	4
2.3	How the Internship was Obtained	4
2.4	Internship Details	4
2.5	Roles and Responsibilities	4
2.6	Technologies Used	5
2.7	Use of AI Assistance	5
2.8	Background: Power Integrity in One Page	5
2.9	Work Overview and Architecture	5
3	Internship Certificate	7
4	Monthly Reports	8
5	Weekly Reports	9
5.1	Week 1 (1 June to 5 June 2026)	9
5.2	Week 2 (8 June to 12 June 2026)	9
5.3	Week 3 (15 June to 19 June 2026)	10
5.4	Week 4 (22 June to 26 June 2026)	10
5.5	Week 5 (29 June to 3 July 2026)	11
5.6	Week 6 (6 July to 10 July 2026)	11
5.7	Week 7 (13 July to 17 July 2026)	12
5.8	Week 8 (20 July to 24 July 2026)	12
5.9	Week 9 (27 July to 31 July 2026)	13
6	Technical Deliverables	14
6.1	Design Under Analysis	14
6.2	Distributed Voltus Report Combiner	14
6.3	Cell-Count / DSP Switching Cross-Check	17
6.4	Distributed Comparison Utility with Outlier Plotting	17
6.5	Voltus–Tempus–Innovus ECO Study	19
6.6	Voltus Instance-Power Merge Bug	19
7	Conclusion	20
7.1	Primary Objectives	20
7.2	Implementation	20
7.3	Challenges Faced	20
7.4	Outcomes	20
7.5	Limitations and Future Scope	21
8	Ending Note	21
9	References Used	21

1. Acknowledgement

I thank my **reporting manager** and the **Voltus Product Engineering team** at Cadence Design Systems, Noida, for a second training stint with the team, for real production problems to work on, and for fast, honest feedback on every script I shipped.

Special thanks to the teammates who reviewed my code line by line, shared legacy run decks, and pushed each tool from “works on my run” to “works on a customer database”. I also thank the HR and IT teams for a smooth onboarding and offboarding.

I am grateful to **Delhi Technological University** for making industrial training part of the curriculum, and to the Department of Applied Physics for the grounding in devices, circuits and computation that this work relied on.

I acknowledge the use of AI assistants, **Claude** and **Gemini**, for drafting and debugging code. Every suggestion was reviewed, tested on real Voltus runs and adapted by me before use.

ISHAAN SINGHAL

(23/EP/047)

2. Introduction

2.1 About the Organization

Cadence Design Systems is a global leader in Electronic Design Automation (EDA). Its software, hardware and IP are used to design and sign off integrated circuits, systems-on-chip (SoCs) and PCBs. Flagship tools span the full flow: Virtuoso (custom/analog), Genus (synthesis), Innovus (place & route), Tempus (static timing) and Voltus (power integrity). Cadence India, with major centres in Noida and Bengaluru, hosts large R&D and product engineering teams for these tools.

2.2 About the Team

The internship was with **Product Engineering (PE) – Voltus**. Voltus is Cadence's power-integrity signoff solution: cell-level power analysis, static and dynamic IR drop, and electromigration (EM) on full-chip designs. The PE team sits between R&D and customers:

- validates Voltus on real and customer designs (accuracy, runtime, capacity);
- tunes run decks and distributed (multi-host) execution;
- reproduces customer issues and files minimal test cases with R&D;
- builds automation around the tool's large, multi-GB report outputs.

2.3 How the Internship was Obtained

I had interned with the same team in June–July 2025, building a Voltus power-verification flow and scripts to consolidate scattered missing-data reports. Based on that work, I was taken back for a two-month industrial training in 2026.

2.4 Internship Details

- **Organization:** Cadence Design Systems (India) Pvt. Ltd., Noida SEZ
- **Duration:** 1st June 2026 to 31st July 2026 (two months)
- **Role:** Product Engineer Intern
- **Mode:** On-site
- **Team:** Product Engineering – Voltus (Power Integrity Signoff)
- **Mentor:** Reporting Manager, Voltus Product Engineering
- **Work:** Distributed automation for Voltus signoff reports; IR-drop ECO study

2.5 Roles and Responsibilities

- Run and debug Voltus power and rail (IR) analysis on RAK (90 nm) and 55 nm designs.
- Tune run.tcl settings; debug local vs. distributed (LSF) execution.
- Build distributed, multithreaded Python tools for Voltus report post-processing.
- Deploy and validate the tools on a customer (QCOM) database; fix LSF submission issues.
- Document tools (review deck, usage guide) and report tool bugs with minimal repro cases.

2.6 Technologies Used

Area	Tools / Techniques
EDA tools	Cadence Voltus (25.13), Tempus, Innovus
Languages	Python 3.7, Tcl, C++ (inline, JIT-compiled at runtime), csh/bash
Compute	IBM LSF (bsub) multi-host grid, NFS shared storage, RHEL Linux
Parallelism	ThreadPoolExecutor, multiprocessing, mmap, GNU sort -parallel
Data formats	LEF/DEF, Liberty, SPEF, TWF, VCD, .iv, .ptiavg, .totalcurrent, JSON, gzip
Visualisation	Matplotlib (headless Agg), Plotly
Docs & workflow	GVim, NoMachine, LaTeX Beamer, PowerPoint, e-mail reviews
AI assistance	Claude, Gemini (drafting, refactoring, debugging)

2.7 Use of AI Assistance

Claude and Gemini were used as coding assistants: drafting boilerplate (argument parsing, logging, config loading), explaining Voltus/LSF error logs, and brainstorming edge cases. Output was treated as a first draft. Algorithms, scheduling logic and all validation were done by me on real Voltus data, and the final code was reviewed by teammates before hand-over.

2.8 Background: Power Integrity in One Page

A chip’s power-grid (PG) network has finite resistance and inductance, so the supply seen by each cell is lower than nominal. Voltus computes this drop for every instance.

$$\begin{aligned}
 \text{Static IR: } \Delta V &= I_{\text{avg}} R_{\text{PG}} \\
 \text{Dynamic IR: } \Delta V(t) &= I(t) R_{\text{PG}} + L \frac{dI}{dt} \\
 \text{Cell power: } P &= \underbrace{\alpha C_L V_{DD}^2 f}_{\text{switching}} + P_{\text{internal}} + \underbrace{V_{DD} I_{\text{leak}}}_{\text{leakage}}
 \end{aligned}$$

Voltus reports the effective voltage across each cell as $V_{\text{DIV}} = V_{\text{PWR}} - V_{\text{GND}}$. Example from the 55 nm design (.iv report, Figure 9): $1.29176 - 0.02628 = 1.26548 \text{ V}$ against a nominal 1.32 V, i.e. an effective drop of 54.5 mV (4.1%). The switching-activity factor α is why the cell-count / switching-file cross-check (Section 6.3) matters: a cell missing from the switching data gets the wrong power, and hence the wrong IR.

2.9 Work Overview and Architecture

All work orbits one fact: in distributed mode Voltus splits the chip into partitions and writes every report per partition (part_0/, part_1/, ...). Figure 1 shows how the three tools I built consume these outputs.

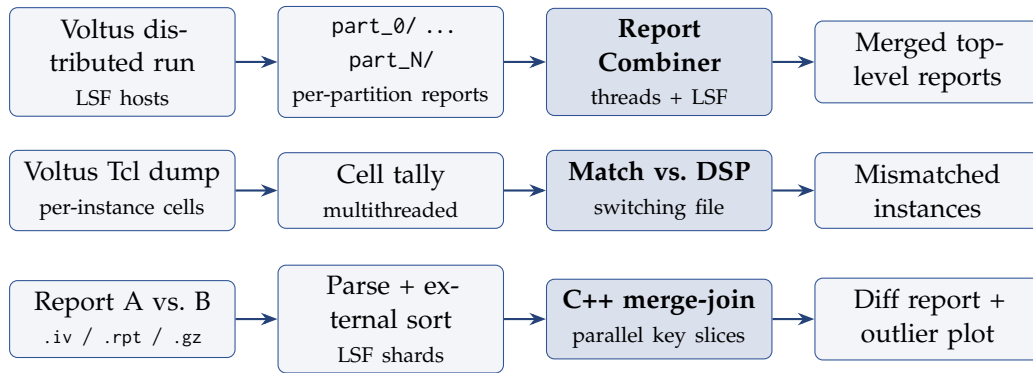


Figure 1: The three automation tools built around Voltus distributed outputs

#	Deliverable	Key techniques
1	Distributed Voltus Report Combiner	LPT load balancing, LSF dispatch, thread pools, C++ mmap k-way merge
2	Cell-Count / DSP Cross-Check	Tcl instance dump, multithreaded chunked parsing, set matching
3	Distributed Comparison Utility	composite keys, external sort, parallel C++ merge-join, outlier plots
4	Voltus–Tempus–Innovus ECO Study	IR hotspot extraction, STA, physical ECO, post-ECO recheck
5	Voltus Merge-Bug Report	data-integrity check, minimal reproducible case

3. Internship Certificate



July 31, 2026

Training Certificate

To Whomsoever It May Concern

This is to certify that **Ishaan Singhal** completed him training with Cadence Design Systems (India) Private Limited from **June 1, 2026**, till **July 31, 2026**.

We wish him success in all future endeavors.

for Cadence Design Systems (India) Pvt. Ltd,



Mohit Kapoor
Sr. HR Manager

Cadence Design Systems (India) Pvt Ltd

Plot Nos. 57 A, B & C, Noida Special Economic Zone, P.O. NEPZ, Noida – 201 305, U.P., India
Phone: +91- 120- 4308300, 5084500, Fax: +91- 120- 5084501
CIN: U30007KA1986PTC080807 Web: www.cadence.com Email: cadenceindia@cadence.com
Registered Office: Level 1, 2 & 3 Campus 4A & 4B, RMZ Ecoworld, Sarjapur - Marathahalli Outer Ring Road, Bengaluru – 560 103



Supporting use of
Recycle Resources

4. Monthly Reports

First Month (June, 2026)

Onboarding, Voltus fundamentals and the first two production scripts. After setting up the VM and tool environment, I ran static IR analysis on the 90 nm RAK and the 55 nm *racerunner* design, then debugged why distributed runs stayed on local CPUs and why rail analysis failed only in distributed mode. The second half went into scripting.

Major tasks:

- VM, tool and LSF environment setup; study of legacy run decks.
- Static IR runs (90 nm RAK, 55 nm *racerunner*); `run.tcl` runtime tuning.
- Local vs. distributed debug: power analysis OK in both, rail analysis fails distributed.
- Missing-report combiner (multithreaded + LSF), later extended to all report types.
- Cell-count / DSP switching cross-check: built, reviewed, re-targeted to a new run, made distributed, edge-case bug fixed.
- Kick-off of a self-driven Voltus–Tempus–Innovus ECO study.

Outcome: Two working, reviewed scripts that turn hour-scale Voltus post-processing into minutes, plus a solid feel for the Voltus signoff flow.

Second Month (July, 2026)

Review, deployment and hardening. The month opened with a manager-requested review of all scripts (PPT + final code by mail), continued with the ECO study in Innovus, and then moved to deploying the comparison utility on a customer (QCOM) database, where LSF submission bugs were fixed and an outlier-plotting module was added and proven on customer data.

Major tasks:

- Script review deck and final code submission (1–3 July).
- Innovus ECO on IR hotspots, new DEF, Voltus re-run and before/after comparison.
- QCOM deployment: `bsub` command fixes, requirement gaps closed.
- Outlier plot module (relative/absolute bounds, top-N worst) proven on QCOM data.
- Found and documented a Voltus instance-power merge inconsistency for R&D.
- Combiner usage guide, hand-over, laptop return and offboarding.

Outcome: Tools running on customer-scale data with documentation, one tool bug reported upstream, and a complete hand-over to the team.

5. Weekly Reports

5.1 Week 1 (1 June to 5 June 2026)

Focus: Onboarding and environment setup

Day	Work Done
Monday 01-06-2026	Joined Cadence, Noida. Laptop issued; HR/IT onboarding and document signing (NDA, policies). Raised access requests for Linux compute, LSF farm and project disk.
Tuesday 02-06-2026	Manager briefing on the team charter and upcoming work. Started Linux VM setup; remote desktop through NoMachine.
Wednesday 03-06-2026	VM bring-up continued: shell environment, tool paths for Voltus/Tempus/Innovus, license variables, scratch and project directories. Short 8-ball pool break with the team.
Thursday 04-06-2026	Received legacy scripts and run decks from earlier work. Laptop and VM setup completed; tool-launch sanity checks passed.
Friday 05-06-2026	Walked through the legacy scripts; read Voltus basics (power analysis modes, static vs. dynamic IR, EM). Next week assigned: RAK walkthrough and first static IR run.

Takeaway: a clean, reproducible environment is the first deliverable in EDA work.

5.2 Week 2 (8 June to 12 June 2026)

Focus: Voltus hands-on and distributed-run debugging

Day	Work Done
Monday 08-06-2026	Received Voltus .tcl run decks. Studied the RAK (Rapid Adoption Kit) static IR flow: design import (LEF/DEF, Liberty, SPEF) → power analysis → rail analysis → reports.
Tuesday 09-06-2026	First static IR run on the 90 nm RAK design; read power reports and IR-drop maps.
Wednesday 10-06-2026	Static analysis on the 55 nm <i>racerunner</i> design (≈ 6.47 M instances, 1.32 V nominal, multiple HV/LV rails). Noted runtime and memory scaling vs. 90 nm.
Thursday 11-06-2026	Teammate sync: cut runtime by tuning power-analysis mode settings in run.tcl. Manager sync: root-cause why jobs stayed on local CPUs and why the distributed-host (LSF) setup was not taking effect.
Friday 12-06-2026	Presented results: power analysis passes local and distributed (distributed clearly faster); rail analysis fails only in distributed mode, passes locally. Assigned two scripting tasks.

Takeaway: isolate failures per stage (power vs. rail) before blaming the grid.

5.3 Week 3 (15 June to 19 June 2026)

Focus: Missing-report combiner and cell-count script

Day	Work Done
Monday 15-06-2026	Started the “missing-file” script: gather missing-data reports (<code>power.missingdata.*</code> , missing-instance/cell lists) scattered across <code>part_*</code> folders and merge them in one pass, bypassing Voltus’ slow sequential merge.
Tuesday 16-06-2026	Added thread-pool multithreading and LSF (<code>bsub</code>) distribution; tuned per feedback and mailed the manager. Started the second script. Quick win at the pool table.
Wednesday 17-06-2026	Cell-count script: dump per-instance cell data from Voltus (Tcl), tally cell counts, match against the dynamic switching file from DSP generation. Prototype demo to a teammate; feedback noted.
Thursday 18-06-2026	Implemented the teammate’s changes. Multithreaded chunked reader: millions of lines parsed and dumped in seconds.
Friday 19-06-2026	Asked to modify the config for a new Voltus run and re-run the script on its output. Config parameters due Monday; prepared the run area and reviewed the parsing logic.

Takeaway: parallel I/O plus de-duplication turns hour-scale merges into minutes.

5.4 Week 4 (22 June to 26 June 2026)

Focus: Validation on a new run, distribution and hardening

Day	Work Done
Monday 22-06-2026	Created the new config, fired the Voltus run and demoed the script. Team flagged output inconsistent with the new run’s report structure.
Tuesday 23-06-2026	Fixed the parser for the new format; revised script validated on the run. New request: an extra column in the final report for readability.
Wednesday 24-06-2026	Made the script distributed (LSF shards) on top of multithreading, with per-shard logs and a final merge step.
Thursday 25-06-2026	Found and fixed an edge-case bug during testing; script stable. Waiting for the column specification.
Friday 26-06-2026	No task assigned. Planned a self-driven Voltus → Tempus → Innovus ECO study to see the full power/timing signoff loop.

Takeaway: scripts must survive format changes between runs; re-test on every new run.

5.5 Week 5 (29 June to 3 July 2026)

Focus: ECO study kick-off and script review deck

Day	Work Done
Monday 29-06-2026	Baseline Voltus run for the ECO study: IR-drop maps, worst-instance list, hotspot regions.
Tuesday 30-06-2026	Tempus STA on the same design: no setup/hold violations, so IR fixes could proceed without timing risk.
Wednesday 01-07-2026	Manager requested a review of all scripts: final code plus working explained in a PPT, by mail. Started code review and clean-up.
Thursday 02-07-2026	Built the deck: problem, architecture, config, usage and logs for the combiner and cell-count scripts; runtime gains from multithreading + distribution.
Friday 03-07-2026	Final pass; mailed the PPT and final scripts to the manager.

Takeaway: explaining code to others forces a cleaner architecture.

5.6 Week 6 (6 July to 10 July 2026)

Focus: Innovus ECO and post-ECO recheck

Day	Work Done
Monday 06-07-2026	Imported the design into Innovus. ECO on IR hotspots (PG reinforcement, decap insertion, cell spreading); exported the modified DEF.
Tuesday 07-07-2026	Voltus re-run on the ECO DEF; before/after IR comparison showed the localized hotspots reduced.
Wednesday 08-07-2026	Set up a Tempus recheck on the post-ECO database. Noted trade-offs: decap area vs. leakage, cell spreading vs. wirelength.
Thursday 09-07-2026	Wrote flow notes; cleaned run directories and logs to free disk quota. End-of-week pool session with the team.
Friday 10-07-2026	Refresh Day – company-wide day off.

Takeaway: an IR fix is a physical change; every ECO needs a power and a timing recheck.

5.7 Week 7 (13 July to 17 July 2026)

Focus: Deployment on the customer (QCOM) database

Day	Work Done
Monday 13-07-2026	New assignment: validate the comparison script (and, in parallel, the combiner) on the QCOM customer database of .iv signoff reports.
Tuesday 14-07-2026	First runs: parsing fine, but LSF job submission failed with bsub command errors under the customer farm settings. Debugged submission strings and shard logs.
Wednesday 15-07-2026	Fixed bsub command construction (queue, resource-string formatting, memory and host span); shards dispatched correctly. Combiner tweaks in parallel.
Thursday 16-07-2026	Runs completed, but not all requirements were met. New task: plot the outliers of the comparison and prove it on QCOM data.
Friday 17-07-2026	Defined the outlier rule (relative % and absolute limits) and top-N worst extraction; started streaming extraction from the comparison report.

Takeaway: customer environments expose assumptions hidden in local runs.

5.8 Week 8 (20 July to 24 July 2026)

Focus: Outlier plotting and proof on QCOM data

Day	Work Done
Monday 20-07-2026	Built the plot module: Value-1 vs. Value-2 scatter, $y = x$ reference, $\pm r\%$ / $\pm a$ limit lines, outliers highlighted; headless Matplotlib (Agg) PNG at 250 dpi.
Tuesday 21-07-2026	Tested locally on a racerunner .iv pair (≈ 6.47 M instances); tuned marker size/alpha for millions of points; bounded min-heap for top-N.
Wednesday 22-07-2026	Hooked the plot into all execution paths (local, grid, low-memory); ran on the QCOM database through LSF.
Thursday 23-07-2026	Cross-checked plotted outliers against raw rows; submitted plots and scripts to the manager.
Friday 24-07-2026	Validated combiner output against Voltus' own merged power.instpwr.rpt.gz: found instance-level mismatches between partition and top-level values.

Takeaway: a plot is proof only when every point traces back to a raw row.

5.9 Week 9 (27 July to 31 July 2026)

Focus: Bug report, documentation and hand-over

Day	Work Done
Monday 27-07-2026	Root-caused the mismatch to Voltus' own merge, not the script: one decap instance shows 7.578e-14 in part_0 but 4.198e-14 at top level. Built a minimal reproducible case.
Tuesday 28-07-2026	Prepared the bug deck for the tool team: Voltus 25.13-s099_1, run script, zgrep evidence (Section 6.6).
Wednesday 29-07-2026	Wrote the combiner "Usage and Output Guide" (Beamer): run commands, config, outputs, logs, sample run.
Thursday 30-07-2026	Final clean-up; handed over scripts, configs and guides; closing review with the manager.
Friday 31-07-2026	Last day: backed up work to the team area, returned the laptop to IT, completed HR offboarding. Farewell game of pool.

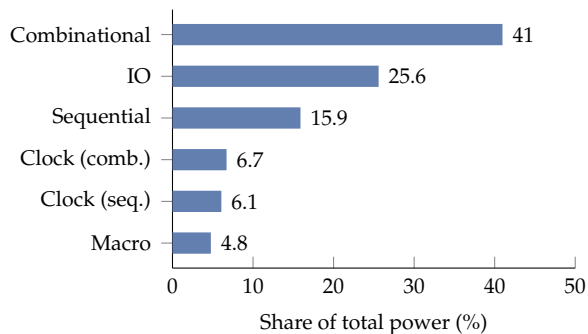
Takeaway: a bug report is only as good as its smallest reproducible case.

6. Technical Deliverables

6.1 Design Under Analysis

Most runs used the 55 nm *racerunner* design. Key numbers, read directly from Voltus outputs (Figures 2 and 9):

Instances	6,470,422
Nominal supply	1.32 V (VDD_LV_COR)
Total power	3832.6 mW
Internal	45.94%
Switching	41.69%
Leakage	12.37%
Partitions	3 (part_0-part_2)



```

142 *
143 -----
144
145
146 Total Power
147 -----
148 Total Internal Power: 1760.68962267 45.9400%
149 Total Switching Power: 1597.68621102 41.6869%
150 Total Leakage Power: 474.20857964 12.3731%
151 Total Power: 3832.58441328
152 -----
153
154
155 Group Internal Power Switching Power Leakage Power Total Power Percentage
156 Power Power Power Power (%)
157 -----
158 Sequential 420.2 146.9 41.98 609.1 15.89
159 Macro 95.29 11.75 75.55 182.6 4.764
160 IO 686 244.2 50.72 980.9 25.59
161 Physical-Only 0 0.001299 0.2724 0.2737 0.00714
162 Combinational 375 925.2 270 1570 40.97
163 Clock (Combinational) 69.64 168.5 18.96 257.1 6.709
164 Clock (Sequential) 114.6 101.1 16.72 232.5 6.065
165 -----
166 Total 1761 1598 474.2 3833 100
167 -----
168
169
170 Clock Internal Power Switching Power Leakage Power Total Power Percentage
171 Power Power Power Power (%)
172 -----
173 enet_mdio_mdc_clk 10.03 3.412 0.4695 13.91 0.3629
174 dspil_slave_sck_port_1_clk_v 4.198e-05 1.95e-05 1.056e-06 6.253e-05 1.632e-06
175 4.198e-05 1.95e-05 1.056e-06 6.253e-05 1.632e-06
176 iic0_clk_PAD_18 5.051 1.682 0.5024 7.235 0.1888
177 enet_rmii_2_mii_rx_clk_PAD_107

```

Figure 2: Voltus power report of racerunner: total and per-group internal, switching and leakage power

6.2 Distributed Voltus Report Combiner

Problem. A distributed Voltus run leaves 100+ report types split across part_* folders (Figure 3). Voltus' own merge is sequential and can take up to an hour. **Goal:** rebuild every top-level report in minutes.

```

dynamic VDD LV COR.ptiavg
dynamic VDD LV IO AURORA.ptiavg
dynamic VDD LV IO MTI.ptiavg
dynamic VDD LV PLL0.ptiavg
dynamic VDD LV PLL1.ptiavg
dynamic VSS HV ADCREF0.ptiavg
dynamic VSS HV ADCREF1.ptiavg
dynamic VSS HV ADCREF2.ptiavg
dynamic VSS HV ADCREF3.ptiavg
dynamic VSS HV ADCW.ptiavg
dynamic VSS HV ATX.ptiavg
dynamic VSS HV IO GPIO1.ptiavg
dynamic VSS HV IO GPIO2.ptiavg
dynamic VSS HV IO GPIO3.ptiavg
dynamic VSS HV IO PDI.ptiavg
dynamic VSS HV IO PWM.ptiavg
dynamic VSS HV PLL0.ptiavg
dynamic VSS HV PLL1.ptiavg
dynamic VSS HV REG3V8.ptiavg
dynamic VSS LV COR.ptiavg
dynamic VSS LV IO AURORA.ptiavg
dynamic VSS LV IO MTI.ptiavg
gui.db/
orig_power_0.options
part_info.json
pm.ob
power_0.cmd
power.clknetworkpwr.rpt
power_inc.json
power.instpwr.rpt
power.lib.rpt
power.meta.json
power.missingdata.lefdef
power.missingdata.lib
power_summary.missingcell.lib.json
power_summary.missinginst.spef.json
power_summary.missinginst.twf.json
power.toggle.rpt
PTIdata/
run.log
run.=OLD-3548580-1=.log'
static default_ground_rail.ptiavg
static default_power_rail.ptiavg
static VDD HV ADCREF1.ptiavg
static VDD HV ADCREF3.ptiavg
static VDD HV ATX.ptiavg
static VDD HV FLA.ptiavg
static VDD HV IO GPIO1.ptiavg
static VDD HV IO GPIO2.ptiavg
static VDD HV IO GPIO3.ptiavg
static VDD HV IO PDI.ptiavg
static VDD HV IO PWM.ptiavg
static VDD HV PMU.ptiavg
static VDD HV REG3V8.ptiavg
static VDD LV COR.ptiavg
static VDD LV IO AURORA.ptiavg
static VDD LV IO MTI.ptiavg
static VDD LV PLL0.ptiavg
static VDD LV PLL1.ptiavg
static VSS HV ADCREF1.ptiavg
static VSS HV ADCREF3.ptiavg
static VSS HV ATX.ptiavg
static VSS HV IO GPIO1.ptiavg
static VSS HV IO GPIO2.ptiavg
static VSS HV IO GPIO3.ptiavg
static VSS HV IO PDI.ptiavg
static VSS HV IO PWM.ptiavg
static VSS HV PLL0.ptiavg
VDD HV IO PWM.avg.totalcurrent
VDD HV PMU.avg.totalcurrent
VDD HV REG3V8.avg.totalcurrent
VDD LV COR.avg.totalcurrent
VDD LV IO AURORA.avg.totalcurrent
VDD LV IO MTI.avg.totalcurrent
VDD LV PLL0.avg.totalcurrent
VDD LV PLL1.avg.totalcurrent
vltus.db/
vltus.pgrail.map
vltus.power.boundaryload
vltus.power.cellstat
vltus.power.cell_type.rpt
vltus.power.large_arrival_time_nets.rpt
vltus.power.missing_netnames.rpt
vltus.power.ptifiles
vltus.power.slewloadoutofrange
VSS HV ADCREF0.avg.totalcurrent
VSS HV ADCREF1.avg.totalcurrent
VSS HV ADCREF2.avg.totalcurrent
VSS HV ADCREF3.avg.totalcurrent
VSS HV ADCW.avg.totalcurrent
VSS HV ATX.avg.totalcurrent
VSS HV IO GPIO1.avg.totalcurrent
VSS HV IO GPIO2.avg.totalcurrent
VSS HV IO GPIO3.avg.totalcurrent
VSS HV IO PDI.avg.totalcurrent
VSS HV IO PWM.avg.totalcurrent
VSS HV PLL0.avg.totalcurrent
VSS HV PLL1.avg.totalcurrent
VSS HV REG3V8.avg.totalcurrent
VSS LV COR.avg.totalcurrent
VSS LV IO AURORA.avg.totalcurrent
VSS LV IO MTI.avg.totalcurrent

```

Figure 3: Contents of one partition folder (part_0/): power, IR (.ptiavg), current waveforms (.totalcurrent), JSON stats and missing-data reports

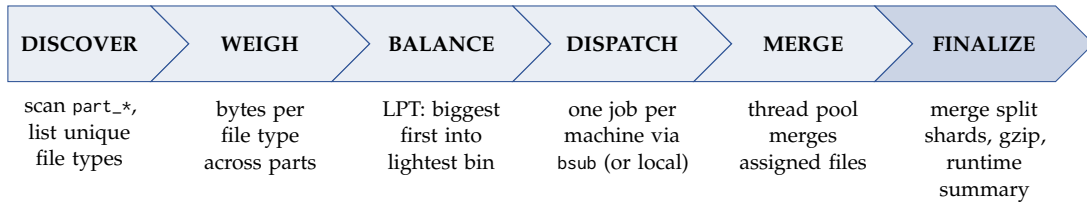


Figure 4: Combiner pipeline

Merge strategy per file type (routed automatically by file name):

File type	Merge rule
*.totalcurrent	sum current per aligned time step (waveforms)
*.json (stats)	recursive dictionary merge, numeric counters summed
missing-inst / missing-cell JSON	streaming, disk-backed de-dup of [instance, cell] pairs (no OOM)
power.instpwr / power.rpt	structured merge; "Total Power" blocks re-summed
power.pgnet.detailed.rpt.gz	grouped by rail; SI-unit parse/format; chip totals recomputed
other .gz and text	sort -u -parallel de-dup + C++ mmap k-way merge

Scheduling knobs.

- **Auto-local:** input below auto_local_gb (5 GB) runs locally; grid queue overhead would be slower.
- **Profiles:** light / heavy LSF profiles pick cores, memory and shard count by input size.
- **Heavy-file split:** files above heavy_split_gb are prepared per partition in parallel, then merged in a finalize phase.
- **Observability:** master.log + worker_N.log in a hidden .work_files/ folder.

Listing 1: Trimmed combine.cfg

```
[combine]
search_dir = <run>/reports/power          # parent of part_* folders
out_dir    = <run>/reports/power/combined
grid       = 1                          # 1 = LSF farm, 0 = local

[default]
auto_local_gb = 5.0

[profile:light]
max_gb = 40.0
shard_count = 4
cores = 8
finalize_cores = 16
bsub_base_cmd = bsub -q lnx64 -W 120 -R "rusage[mem=64000] span[hosts=1]"
```

Result on a real run (from the runtime summary, Figure 5):

Partitions / unique file types	3 / 115
Total input	1.951 GB
Workers	4 (96 local threads each)
Fastest / slowest worker	11.3 s / 197.6 s
Total wall time	3 min 17 s (vs. up to ~1 h sequential)

```
ishaans@jfdsp179 ~/scratch/Main_Racerunner_Files/Run_for_combining_script/combination_script/new %
>> python3 /home/ishaans/scratch/Main_Racerunner_Files/Run_for_combining_script/Combination_script/v3_combine_all_reports.py
=====
VOLTUS REPORT COMBINER - ALL FILE TYPES (distributed)
=====
SEARCH DIR : /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power
OUT DIR    : /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports
GRID       : True  MACHINES: 4  STRATEGY: file
CONFIG     : /home/ishaans/scratch/Main_Racerunner_Files/Run_for_combining_script/Combination_script/combine.cfg

[DISCOVER] 3 part dirs | 115 unique file types | 1.951 GB total input data
[AUTO] Data is only 1.95 GB (threshold 5.0 GB).
[AUTO] Switching to LOCAL mode - grid queue overhead would be slower.
[AUTO] To force grid anyway: setenv FORCE_GRID 1

[BALANCE] Work distribution across 4 machine(s):
Machine 0: 28 items ( 0 split-part), 0.536 GB
Machine 1: 29 items ( 0 split-part), 0.511 GB
Machine 2: 29 items ( 0 split-part), 0.459 GB
Machine 3: 29 items ( 0 split-part), 0.445 GB
[SCHED] Per-worker allocations:
Worker 0: ~0.536 GB, local_threads=96, grid_cores=16
Worker 1: ~0.511 GB, local_threads=96, grid_cores=16
Worker 2: ~0.459 GB, local_threads=96, grid_cores=16
Worker 3: ~0.445 GB, local_threads=96, grid_cores=16
[MASTER] split manifest -> /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/.work_files/split_manifest.json (0 entries)
[MASTER] Worker 0 launched (pid 1817591) -> /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/.work_files/worker_0.log
[MASTER] Worker 1 launched (pid 1817592) -> /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/.work_files/worker_1.log
[MASTER] Worker 2 launched (pid 1817593) -> /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/.work_files/worker_2.log
[MASTER] Worker 3 launched (pid 1817594) -> /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/.work_files/worker_3.log
[MASTER] Worker 0 FINISHED OK - elapsed 11.3s (0.2 min)
[MASTER] Worker 1 FINISHED OK - elapsed 45.1s (0.8 min)
[MASTER] Worker 2 FINISHED OK - elapsed 197.6s (3.3 min)
[MASTER] Worker 3 FINISHED OK - elapsed 197.6s (3.3 min)
[MASTER] Master log written -> /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/.work_files/master.log

=====
RUNTIME SUMMARY
=====
Worker 0 : 11.3s ( 0.2 min) ~0.536 GB
Worker 1 : 45.1s ( 0.8 min) ~0.511 GB
Worker 2 : 197.6s ( 3.3 min) ~0.459 GB
Worker 3 : 197.6s ( 3.3 min) ~0.445 GB
=====
Slowest worker : 197.6s | Fastest worker : 11.3s | Skew : 186.3s
Dispatch phase (shard/merge workers) : 197.6s ( 3.3 min)
Finalize phase (shard merge/compress): 0.0s ( 0.0 min)
=====
TOTAL WALL TIME : 00:03:17 (197.6s)
=====
SUCCESS - combined Files are in: /icd/rd_t3nb_004/ishaans/Main_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports/
=====
```

Figure 5: Combiner run: discovery, auto-local decision, load balance across 4 workers, runtime summary

```

ishaans@sjv1-ishaans /icd/rd_t3nb_004/ishaans/Plain_Racerunner_Files/Run_for_combining_script/new_reports_none/power/recheck_combined_reports %
>> ls
cellIDMap                                dynamic_VSS_HV_ADCREF1.ptiavg          power_inc.json                         VDD_HV_ADCREF3.avg.totalcurrent       voltus_power.stateprop.sourcesstats.gz
def_geo_part_mapping.json               dynamic_VSS_HV_ADCREF2.ptiavg          power_instpw.rpt.gz                   VDD_HV_ATX.avg.totalcurrent           voltus_power.stateprop.stats.gz
dynamic_generalVDD.ptiavg               dynamic_VSS_HV_ADCREF3.ptiavg          power.lib.rpt.gz                      VDD_HV_FLTA.avg.totalcurrent          voltus_power.stateprop.stats.json
dynamic_generalVSS.ptiavg               dynamic_VSS_HV_ATX.ptiavg              power.missingdata.lib.parser.log       VDD_HV_I0_GPT01.avg.totalcurrent      voltus_power.stateprop.switchsrc.gz
dynamic_VDD_HV_ADCREF0.ptiavg           dynamic_VSS_HV_I0_GPT01.ptiavg         power.missingdata.spef.parser.log      VDD_HV_I0_GPT02.avg.totalcurrent      voltus_power.togglestats
dynamic_VDD_HV_ADCREF1.ptiavg           dynamic_VSS_HV_I0_GPT02.ptiavg         power.missingdata.twf.parser.log       VDD_HV_I0_GPT03.avg.totalcurrent      VSS_HV_ADCREF0.avg.totalcurrent
dynamic_VDD_HV_ADCREF2.ptiavg           dynamic_VSS_HV_I0_GPT03.ptiavg         power.missingdata.vcd.json            VDD_HV_I0_PDI.avg.totalcurrent        VSS_HV_ADCREF1.avg.totalcurrent
dynamic_VDD_HV_ADCREF3.ptiavg           dynamic_VSS_HV_I0_PDI.ptiavg           power.pgnnet.detailed.rpt.gz          VDD_HV_I0_P4M.avg.totalcurrent        VSS_HV_ADCREF2.avg.totalcurrent
dynamic_VDD_HV_ATX.ptiavg               dynamic_VSS_HV_I0_P4M.ptiavg           power.pgnnet.rpt.gz                  VDD_HV_P4M.avg.totalcurrent           VSS_HV_ADCREF3.avg.totalcurrent
dynamic_VDD_HV_FLTA.ptiavg              dynamic_VSS_HV_PLI0.ptiavg             power.rpt.gz                          VDD_HV_REG3V8.avg.totalcurrent        VSS_HV_ATX.avg.totalcurrent
dynamic_VDD_HV_I0_GPT01.ptiavg          dynamic_VSS_HV_PLI1.ptiavg             power.stats.setpower.gz               VDD_HV_REG3V8.avg.totalcurrent        VSS_HV_I0_GPT01.avg.totalcurrent
dynamic_VDD_HV_I0_GPT02.ptiavg          dynamic_VSS_HV_REG3V8.ptiavg           power_summary.annotatedcell.lib.json  VDD_LV_I0_AURORA.avg.totalcurrent     VSS_HV_I0_GPT02.avg.totalcurrent
dynamic_VDD_HV_I0_GPT03.ptiavg          dynamic_VSS_LV_COR.ptiavg              power_summary.json                   VDD_LV_I0_MTI.avg.totalcurrent        VSS_HV_I0_GPT03.avg.totalcurrent
dynamic_VDD_HV_I0_PDI.ptiavg            dynamic_VSS_LV_I0_AURORA.ptiavg        power_summary.missingcell.lib.json    VDD_LV_PLI0.avg.totalcurrent          VSS_HV_I0_PDI.avg.totalcurrent
dynamic_VDD_HV_I0_P4M.ptiavg            dynamic_VSS_LV_I0_MTI.ptiavg           power_summary.missinginst.spef.json   VDD_LV_PLI1.avg.totalcurrent          VSS_HV_I0_P4M.avg.totalcurrent
dynamic_VDD_HV_P4M.ptiavg               orig_power_0.options                  power_summary.missinginst.twf.json    voltus_pgnrail_map                   VSS_HV_PLI0.avg.totalcurrent
dynamic_VDD_HV_REG3V8.ptiavg            orig_power_1.options                  power_togglelate.rpt.gz               voltus_power.boundaryload             VSS_HV_PLI1.avg.totalcurrent
dynamic_VDD_LV_COR.ptiavg               orig_power_2.options                  run.log                               voltus_power.cellstat                 VSS_HV_REG3V8.avg.totalcurrent
dynamic_VDD_LV_I0_AURORA.ptiavg         part_info.json                       stat.txt                              voltus_power.cell_type.rpt            VSS_LV_COR.avg.totalcurrent
dynamic_VDD_LV_I0_MTI.ptiavg            pw_0                                  timing08_rail_top                     voltus_power.large_arrival_time_nets.rpt VSS_LV_I0_AURORA.avg.totalcurrent
dynamic_VDD_LV_PLI0.ptiavg              power_0.cmd                           VDD_HV_ADCREF0.avg.totalcurrent       voltus_power.missing_netnames.rpt.gz  VSS_LV_I0_MTI.avg.totalcurrent
dynamic_VDD_LV_PLI1.ptiavg              power_1.cmd                           VDD_HV_ADCREF1.avg.totalcurrent       voltus_power.ptifiles                 VSS_LV_I0_PDI.avg.totalcurrent
dynamic_VSS_HV_ADCREF0.ptiavg            power_2.cmd                           VDD_HV_ADCREF2.avg.totalcurrent       voltus_power.slewloadoutofrange

```

Figure 6: Consolidated top-level output directory produced by the combiner

6.3 Cell-Count / DSP Switching Cross-Check

Problem. Every instance must appear in the dynamic switching file produced during DSP generation; a missing or extra instance silently skews switching power (α) and IR. **Approach:**

- Tcl: tuned DSP/power-analysis settings; dump per-instance cell data from Voltus.
- Python: multithreaded chunked reader; tally cell counts per cell type.
- Match against the switching file; flag missing, extra and count-mismatched instances.
- Re-targeted to a new run config (Week 4), made LSF-distributed, edge case fixed.

Outcome: a one-command coverage check that reads and dumps millions of lines in seconds; an extra readability column was requested and left pending spec at hand-over.

6.4 Distributed Comparison Utility with Outlier Plotting

Problem. Compare two signoff reports instance by instance (e.g. two .iv IR reports, each ≈ 6.47 M lines, Figure 10) and show where and how much they differ. Scale target: billion-row inputs on a customer database.

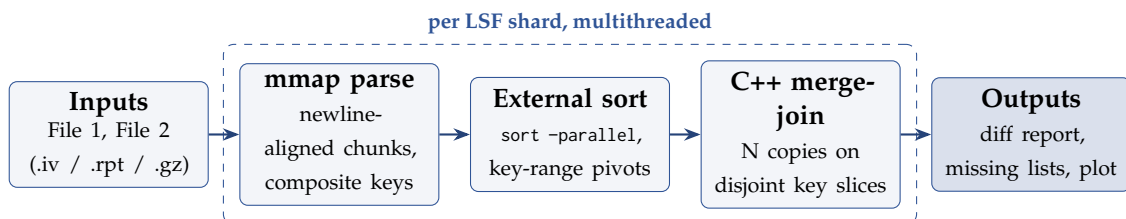


Figure 7: Comparison pipeline

Key features.

- Any column pair: user selects instance column(s) and value column per file; composite keys like 0, 2; plain or gzip input.
- Output row: Instance | Value1 | Value2 | Diff | Dev%, plus instances missing on either side.
- LSF sharding by gb_per_shard; light/heavy profiles; low-memory mode on by default.
- Inline C++ merge-join compiled at runtime, run in parallel on key-range slices, so the finalize step uses all allocated cores.

Listing 2: Typical invocation (IR comparison of two runs)

```
python3 comparison_script_distributed.py \
--file1 runA/VDD_LV_COR_VSS_LV_COR.avg.iv --instcol1 1 --valcol1 2 \
--file2 runB/VDD_LV_COR_VSS_LV_COR.avg.iv --instcol2 1 --valcol2 2 \
-r 5 -a 0.01 -n 100 --outdir cmp_out
```

Outlier rule. For each matched instance with values v_1, v_2 :

$$\text{diff} = v_1 - v_2, \quad \text{dev} = \frac{v_1 - v_2}{v_2} \times 100\%, \quad \text{outlier} \iff |\text{dev}| > r \vee |\text{diff}| > a.$$

The N worst rows are kept in a bounded min-heap ($O(M \log N)$ for M rows) and written to a summary; all points are drawn in one static PNG (Figure 8).

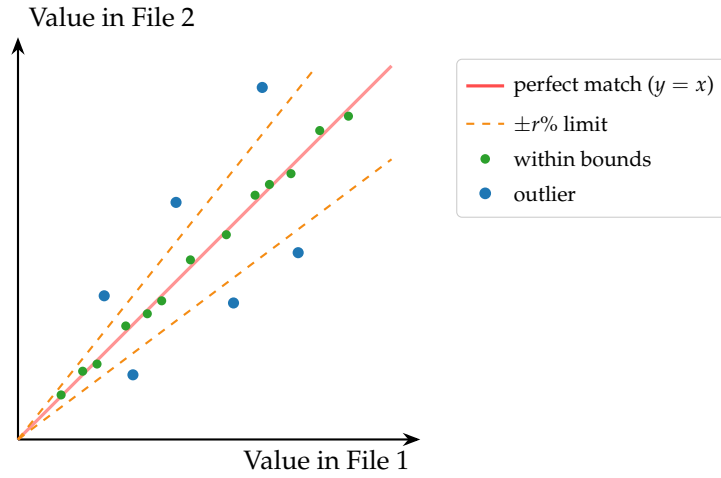


Figure 8: Outlier plot layout (schematic). Real plots carry millions of points.

```
1 VERSION      "3.0"
2 CREATION     "Tue May 26 02:36:00 2020"
3 CREATOR      "sidkumar"
4 PROGRAM      "VOLTUS"
5 DIVIDERCHAR   "/"
6 DESIGN       "racerunner"
7 UNITS        "VOLTAGE VOLT 1"
8 INSTANCE_COUNT 6470422
9 NOMINAL_VOLTAGE 1.32
10 POWER_NET    VDD_LV_COR
11 GROUND_NET    VSS_LV_COR
12 WINDOW       NA
13 RP_VALUE     IV/DIV
14 RP_FORMAT     DETAIL
15 RP_INST_LIMIT 2000000000
16 RP_THRESHOLD -1e+06
17 RP_PIN_NAME   FALSE
18 MICRON_UNITS 10000
19
20 INST_NAME DIV PWR IV GND IV CELL NAME
21 "NA" means the instance is disconnected from the net or the instance does not have timing/switching window
22 BEGIN
23 safety_lake/chk_2427b/lbist_chk_2427b/z7_core_b/cpu_nex1/cpu/cpu_dpath/z8_zmac2_accumulate/p1646A 1.46546 1.29175 0.0262
11x6
24 - safety_lake/chk_2427b/lbist_chk_2427b/z7_core_b/cpu_nex1/cpu/cpu_dpath/z8_zmac2_accumulate/p1777A 1.46547 1.29176 0.0262
25 safety_lake/SAFETY_LAKE_DECAP_imp12_63776 1.26548 1.29176 0.02628 H555 LH DECAP5
26 safety_lake/chk_2427b/lbist_chk_2427b/z7_core_b/cpu_nex1/cpu/cpu_dpath/z8_zmac2_accumulate/p8756A 1.26548 1.29176 0.0262
27 safety_lake/SAFETY_LAKE_DECAP_imp12_63777 1.26549 1.29177 0.02628 H555 LH DECAP4
28 safety_lake/chk_2427b/lbist_chk_2427b/z7_core_b/cpu_nex1/cpu/cpu_dpath/z8_zmac2_accumulate/p8687A15989 1.26550 1.29177 0.0262
MUX2111x6
29 - safety_lake/SAFETY_LAKE_DECAP_imp12_63776 1.26551 1.29178 0.02627 H555 LH DECAP6
30 safety_lake/chk_2427b/lbist_chk_2427b/z7_core_b/cpu_nex1/cpu/cpu_dpath/z8_zmac2_accumulate/p1803A 1.26552 1.29178 0.0262
31 safety_lake/SAFETY_LAKE_DECAP_imp12_63475 1.26552 1.29175 0.02622 H555 LH DECAP7
32 safety_lake/SAFETY_LAKE_DECAP_imp12_63775 1.26553 1.29179 0.02626 H555 LH DECAP5
33 safety_lake/SAFETY_LAKE_DECAP_imp12_63474 1.26553 1.29175 0.02622 H555 LH DECAP8
34 - safety_lake/welltap_safety_lake_4694 1.26554 1.29180 0.02626 H555 LH FILLERMMMS
```

Figure 9: .iv IR report: header and per-instance DIV / PWR / GND voltages

```
[kvishnu@cs]cvf-kvishnu Script_run]$ wc -l *
1199 ,
6 big.sh
25 cmp_master.log
102 cmp_shard 0.lsflag
71 cmp_shard 1.lsflag
387 colu.txt
1180 combine_all_v2.py
57 comparison.cfg
6470425 comparison_report.txt
6470038 comparison_report_v3_v3.txt
6470038 comparison_report_v4.txt
1196 comparison_script_distributed_fixed.py
407 epseivplot.pl
1190 gemi_dis.py
wc: gemi_run: Is a directory
0 gemi_run
7 missing_instances.txt
1 neeche.txt
388 removed.txt
387 sort.txt
4 tempc.cpp
22 upar.txt
1292 v3_comparison.py
860 v4.py
6470058 VDD_LV_COR_VSS_LV_COR.avg2.iv
69 VDD_LV_COR_VSS_LV_COR.avg_div.gif
6470445 VDD_LV_COR_VSS_LV_COR.avg.iv
95 VDD_LV_COR_VSS_LV_COR.avg_limit_div.gif
```

Figure 10: Scale: input .iv files and comparison reports of ≈ 6.47 M lines

QCOM deployment. On the customer farm, job submission failed until the bsub command was rebuilt from the profile (queue, runtime limit, select/rusage/span resource string). After the fix, shards dispatched correctly and the outlier plots were proven against the customer data.

6.5 Voltus–Tempus–Innovus ECO Study

Self-driven study during a gap in assigned work: fix IR hotspots physically and prove that power improves without breaking timing.

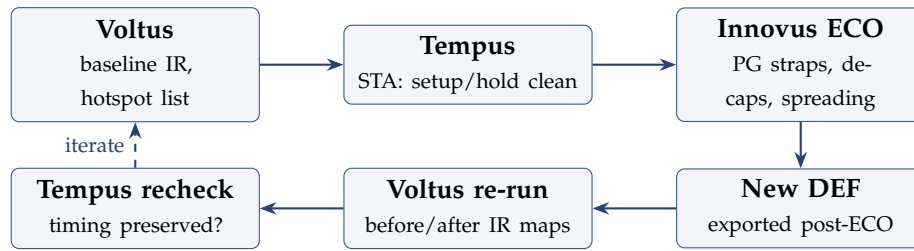


Figure 11: ECO loop: power fix in Innovus, verified by Voltus and Tempus

Step	Tool	Purpose / result
Baseline	Voltus	IR-drop maps; instances with the worst effective voltage
Timing gate	Tempus	no setup/hold violations, so physical changes are safe to try
ECO	Innovus	PG reinforcement, decap insertion, cell spreading near hotspots; new DEF
Re-analysis	Voltus	before/after comparison; localized hotspots reduced
Recheck	Tempus	confirm the ECO did not create timing violations

6.6 Voltus Instance-Power Merge Bug

While validating the combiner, instance power in Voltus' own top-level `power.instpwr.rpt.gz` did not match the per-partition report for the same run (Voltus 25.13-s099_1). The script was ruled out by reproducing the gap with Voltus outputs alone:

```

[ubuntu@sjvdspr179 ~]$ cd /scratch/Main_RaceRunner_Files/RUN_SK/non_distributed_run/correct_correct_reports/power %
>> zgrep "AURORA_DECAP_1000" power.instpwr.rpt.gz part0/power.instp*
power.instpwr.rpt.gz:AURORA_DECAP_1000/vdd 0 0 0 2.05e-06 2.05e-06 4.198e-14 HS55 LS_NDECAP32
power.instpwr.rpt.gz:AURORA_DECAP_1000/vdds 0 1000 0 0 1000 2.048e-05 HS55 LS_NDECAP32
part_0/power.instpwr.rpt.gz:AURORA_DECAP_1000/vdds 0 0 0 1000 0 0 1000 3.697e-05 HS55 LS_NDECAP32
part_0/power.instpwr.rpt.gz:AURORA_DECAP_1000/vdd 0 0 0 0 0 2.05e-06 2.05e-06 7.578e-14 HS55 LS_NDECAP32
  
```

Figure 12: zgrep evidence: same instance, different power in top-level vs. part_0 report

Instance / pin	part_0 report	Top-level merged
AURORA_DECAP_1000/vdd	7.578e-14	4.198e-14
AURORA_DECAP_1000/vdds	3.697e-05	2.048e-05

Delivered to the tool team as a short deck with version, run script and a minimal reproducible case.

7. Conclusion

Two months in Voltus Product Engineering took me from first static IR run to tools running on a customer database. The core lesson: in signoff, speed matters only if the numbers are provably right, so every tool shipped with logs, validation and a way to trace any output back to raw data.

7.1 Primary Objectives

- Cut Voltus post-processing time for distributed runs from hours to minutes.
- Automatically catch coverage gaps between Voltus instance data and switching data.
- Compare billion-row signoff reports and surface outliers visually.
- Understand the Voltus–Tempus–Innovus signoff/ECO loop hands-on.

7.2 Implementation

File	Responsibility	Lines
v3_combine_all_reports.py	distributed combiner, all report types, C++ merge engine	3236
combine.cfg	grid/local switch, LSF profiles	30
comparison_script_distributed.py	sharded comparison, C++ merge-join, outlier plot	1773
comparison.cfg	auto-local threshold, shard size, LSF profiles	42
cell-count script (Tcl + Python)	instance dump, cell tally, switching-file match	–
Review deck, usage guide, bug deck	documentation and hand-over	–

7.3 Challenges Faced

- **Distributed rail analysis:** failed on LSF but passed locally; power stage had to be separated from rail stage to isolate the fault.
- **Changing report formats:** a new run config broke parsing; fixed by format-tolerant parsing and re-testing on every run.
- **Memory:** multi-GB missing-instance JSON caused OOM when loaded whole; replaced by a streaming, disk-backed merge.
- **Merge-join desync:** an early C++ merge re-read an already-consumed line and reported false “missing” instances; fixed by caching the parsed key/value per side.
- **Customer farm:** bsub resource strings that worked internally failed on QCOM; rebuilt from config profiles.
- **Trust in the reference:** Voltus’ own merged report was itself inconsistent; proved with a minimal case before blaming or fixing the script.

7.4 Outcomes

- Report merge: **3 min 17 s** for 115 file types / 1.95 GB vs. up to ~1 h sequential.
- Comparison utility deployed and proven on the QCOM database, with outlier plots.
- Cell-count / DSP cross-check reading and dumping millions of lines in seconds.
- One Voltus data-integrity bug reported to R&D with a reproducible case.
- Review deck, usage guide and full hand-over of code and configs.

7.5 Limitations and Future Scope

- Extra readability column in the cell-count report pending specification.
- ECO study is qualitative; next step is a quantified before/after table (worst IR, hotspot count, WNS/TNS).
- Add a regression suite (golden merged reports) so every change is auto-validated.
- Interactive (Plotly/HTML) outlier view for drill-down on large runs.
- Upstream the combiner logic into the native Voltus merge step.

8. Ending Note

This training bridged classroom physics ($P = \alpha CV^2 f$, $V = IR$) and industrial signoff, where those equations run on six million instances across a compute farm. I learned to build tools for scale, to validate before claiming, and to report problems with evidence. AI assistants made me faster, but understanding, testing and final decisions stayed with me.

I thank the Voltus Product Engineering team at Cadence and Delhi Technological University for the opportunity. And yes, the pool table got a lot of use.

9. References Used

- Cadence Design Systems : <https://www.cadence.com>
- Cadence Voltus, Tempus and Innovus user guides (internal documentation, release 25.1)
- IBM Spectrum LSF documentation : <https://www.ibm.com/docs/en/spectrum-lsf>
- Python concurrent.futures : <https://docs.python.org/3/library/concurrent.futures.html>
- Python mmap : <https://docs.python.org/3/library/mmap.html>
- GNU sort : https://www.gnu.org/software/coreutils/manual/html_node/sort-invocation.html
- Matplotlib : <https://matplotlib.org>
- J. M. Rabaey, A. Chandrakasan, B. Nikolić, *Digital Integrated Circuits*, 2nd ed., Pearson.
- Claude (Anthropic) : <https://claude.ai>
- Gemini (Google) : <https://gemini.google.com>